

14. Une boucle dans une boucle : la grille

Pour chaque ligne, toutes les colonnes : c'est ainsi qu'on remplit un écran.

1. Ce qu'il faut comprendre

Mettre un `for` dans un `for` : la boucle du dedans fait **tous ses tours** pour **chaque** tour de celle du dehors. 4 lignes × 8 colonnes = **32** passages dans le bloc le plus profond.

Suis-la à la main : `ligne` vaut 0, et `colonne` va de 0 à 7 — la première rangée est finie. Puis `ligne` vaut 1, `colonne` **repart de 0** (sa déclaration est rejouée), et ainsi de suite. On lit l'écran comme un livre : de gauche à droite, puis la ligne suivante.

Les noms comptent : `ligne` et `colonne` se relisent mieux que `i` et `j`, et on voit tout de suite que `texte(colonne, ligne, ...)` est dans le bon ordre. Inverser les deux est une erreur classique : le rectangle se retrouve couché.

Le triangle montre la vraie puissance : la **limite du dedans dépend du dehors** — `colonne <= ligne`. La première rangée a 1 case, la deuxième 2, la sixième 6. $1 + 2 + 3 + 4 + 5 + 6 = 21$.

Tout décor de jeu Game Boy est une grille de tuiles de 8×8 pixels : ce double `for` est **la** boucle du dessinateur de niveaux.

À toi : retourne le triangle — la pointe en bas — en changeant seulement la condition du dedans.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **588 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
main()         190 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← ecran() — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase    10 o ← texte() — trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() — écrit un nombre en chiffres
CopierTuiles   40 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles         176 o ← CopierTuiles — 11 lettres sur 43
"#"            1 o ← texte() — un texte gravé
"#"            1 o ← texte() — un texte gravé
```

Lettres gravées : 0123456789#

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 2    52 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
ecran() × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
```

```
nombre() × 2    20 o sur place, dans main() + EcrireNombre (33 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : dessine un rectangle puis un triangle de #, et compte les cases.
//
// Ce qui est nouveau : une boucle DANS une boucle.
// La boucle du dehors choisit la ligne ; pour CHAQUE ligne, la boucle du
// dedans parcourt toutes les colonnes.

#include <ecran>    // éteint ou rallume l'écran
#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    ecran(0);                               // On dessine écran éteint (plus rapide).
    uint8_t cases = 0;                      // Le nombre de # posés.

    // Un rectangle : 4 lignes de 8 colonnes.
    for (uint8_t ligne = 0; ligne < 4; ligne++) { // ligne = 0, 1, 2, 3
        for (uint8_t colonne = 0; colonne < 8; colonne++) { // colonne = 0 à 7, pour chaque ligne
            texte(1 + colonne, 1 + ligne, "#"); // « 1 + » décale d'une case
                                                    // pour ne pas coller au bord.
            cases++;                               // un # de plus.
        }
    }
    nombre(12, 2, cases);                    // 4 lignes × 8 colonnes = 32 : affiche 032.

    // Un triangle : la ligne n a n + 1 cases.
    uint8_t triangle = 0;
    for (uint8_t ligne = 0; ligne < 6; ligne++) { // 6 lignes : 0 à 5
        for (uint8_t colonne = 0; colonne <= ligne; colonne++) { // la colonne va jusqu'à ligne :
                                                                    // ligne 0 → 1 case, ligne 1 → 2 ...
            texte(1 + colonne, 7 + ligne, "#"); // le triangle commence ligne 7.
            triangle++;
        }
    }
    nombre(12, 10, triangle);                // 1 + 2 + 3 + 4 + 5 + 6 = 21 : affiche 021.
    ecran(1);                                // On rallume : tout apparaît.

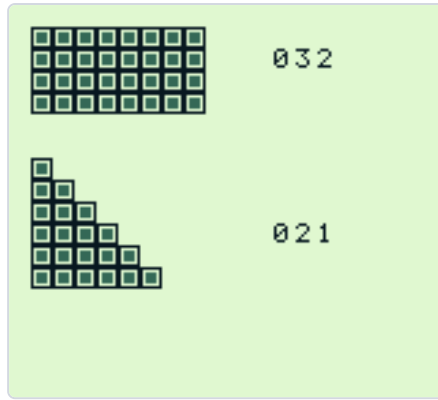
    while (true) {                            // La boucle du jeu.
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un rectangle de 4×8 (032 cases), et en dessous un triangle en escalier (021 cases).

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le rectangle est plein
- ✓ 32 passages dans le bloc du dedans
- ✓ le triangle monte d'une case par ligne
- ✓ $1 + 2 + 3 + 4 + 5 + 6 = 21$

← [13. Compter à l'envers, et le piège du zéro](#)

[sommaire](#)

[15. break et continue : sortir, ou sauter un tour](#) →